

ESTRUCTURAS DE DATOS - 2026-2 Implementación y Análisis de Complejidad de Listas, Pilas y Colas en Java

Estudiante: Marco Antonio García Villasmil

Código: 1094050756

Profesor: David Herrera

Monitora: Angela Camila Siabato Londoño

Repositorio: <https://github.com/marcogarciavillasmil/lab-listas-pilas-colas>

1. Objetivo del Taller Acá tocaba armar en Java la estructura List con listas enlazadas, en sus cuatro variantes: simple sin cola, simple con cola, doble sin cola y doble con cola. También había que hacer Stack y Queue, cada una en dos versiones (arreglo dinámico y arreglo circular), para un total de 8 implementaciones. Con todo compilando y funcionando, lo que realmente importa del taller es medir cuánto se demora cada método según el tamaño de la entrada, comparar eso contra lo que predice la teoría en notación Big-O, y sacar conclusiones de cuál implementación conviene según el caso.

2. Implementación

List (Listas Enlazadas) Para esta parte armé una interfaz común, MyList, con los 8 métodos que pide el enunciado: pushFront, pushBack, popFront, popBack, find, erase, addBefore y addAfter. Las 4 variantes de abajo implementan exactamente lo mismo, solo que manejan los punteros de forma distinta por dentro.

- **SinglyLinkedListNoTail:** La versión más simple de las cuatro, solo guarda un puntero a head. Cualquier operación que necesite llegar al final (pushBack, popBack, topBack) o encontrar el nodo anterior a otro (erase, addBefore) termina recorriendo la lista completa, porque no hay una forma más corta de llegar hasta ahí.
- **SinglyLinkedListWithTail:** Acá agregué un puntero tail, que resuelve pushBack de una (ya no toca recorrer para insertar al final), pero popBack sigue quedando mal: sin puntero al nodo anterior, para actualizar quién queda como nuevo tail toca recorrer igual desde el head.
- **DoublyLinkedListNoTail:** Cada nodo tiene ahora un puntero prev además del next, así que dado cualquier nodo ya se puede hacer erase, addBefore y addAfter en $O(1)$. Lo que queda pendiente: al no tener tail, llegar hasta el último nodo sigue costando $O(n)$.

- **DoublyLinkedListWithTail:** Esta junta las dos cosas (head, tail, prev y next en cada nodo) y es la única que queda en $O(1)$ en casi todo. La excepción es find, que al no haber indexación por posición no tiene otra opción que recorrer.

Algo que decidí desde el principio: find() no devuelve un booleano ni un índice, devuelve directamente la referencia al Node. La idea es no buscar el elemento dos veces: si solo dijera "sí está" o "no está", borrarlo o insertar algo al lado después tocaría recorrer la lista otra vez para ubicarlo, otro $O(n)$ de paso. Al devolver el Node, ese segundo paso queda en pura manipulación de punteros. Por eso en la tabla de 4.1 las complejidades de erase, addBefore y addAfter asumen que ya se tiene el Node en la mano (el que devolvió find()); no incluyen el costo de buscarlo, que en las 4 variantes sigue siendo $O(n)$ porque ninguna tiene indexación directa.

MyStack y MyQueue Para estas dos estructuras terminé probando dos estrategias distintas de arreglo:

- **Arreglo Dinámico (ArrayStack / ArrayQueue):** Es un arreglo interno que se duplica cada vez que se llena, lo mismo que hace ArrayList por dentro. En el Stack esto basta para que push y pop queden en $O(1)$ amortizado, porque las dos operan sobre el tope. Donde se complica es en el Queue: enqueue mete el elemento al final sin drama, pero dequeue tiene que sacar la posición 0 y correr todos los elementos un puesto hacia atrás, lo que deja esa operación en $O(n)$.
- **Arreglo Circular (CircularArrayStack / CircularArrayQueue):** Acá cambié el arreglo plano por uno circular, con un índice front calculado módulo la capacidad. Para el Stack casi no cambia nada, porque sigue operando en un solo extremo; para el Queue sí hace la diferencia: dequeue deja de mover elementos y solo mueve el índice front, quedando en $O(1)$.

Para el crecimiento del arreglo usé la estrategia típica de amortización: cuando se llena, se duplica la capacidad (capacidad $\ast = 2$). Puede que una inserción puntual salga cara ($O(n)$ por el redimensionamiento), pero si se ve el costo total de n inserciones, sigue siendo $O(n)$.

3. Metodología de Medición Para medir usé System.nanoTime() y reporté todo en microsegundos, porque con milisegundos y listas chiquitas todo se redondeaba a 0 y no se notaba la diferencia entre $O(1)$ y $O(n)$. La mecánica para cada tamaño n : armar la estructura con esos n elementos usando un método $O(1)$ (para no distorsionar lo que quiero medir), y luego repetir el método varias veces y sacar el promedio. En push/pop deshago cada operación justo después de medirla, para que el tamaño no cambie a mitad del experimento. En los métodos de búsqueda (find, erase, addBefore, addAfter, delete) el objetivo se elige al azar, con semilla fija en 42 para que sea reproducible. Los tamaños fueron $n = 10, 100, 1000, 10\ 000, 100\ 000$ y $1\ 000\ 000$; al principio quería llegar hasta 10^8 como sugiere el enunciado, pero una sola corrida de pushBack en SinglyNoTail con $n=10^6$ ya se demoraba varios minutos -con 10^8 el laboratorio no se terminaba nunca-, así que me quedé en 10^6 : en las gráficas ya se ve clarísima la tendencia (recta que sube

en log-log para $O(n)$, línea plana para $O(1)$), y subir más solo iba a aumentar el tiempo de espera, no a cambiar la conclusión.

Vale la pena aclarar que no usé la misma cantidad de repeticiones en todos los tamaños. Para pushFront/pushBack/popFront/popBack/find usé 300 repeticiones en $n=10$, 100 y 1000, bajando a 150 en $n=10\ 000$, 50 en $n=100\ 000$ y apenas 15 en $n=1\ 000\ 000$ (si no, la corrida se demoraba horas en los métodos $O(n)$ con n grande). Para erase/addBefore/addAfter/delete, ya más costosos de por sí, usé solo 5 repeticiones en $n=10$ y $n=100$. Esto importa porque con pocas repeticiones el promedio queda más expuesto al ruido: la JVM no alcanza a calentar (el JIT necesita más invocaciones antes de compilar a código nativo), y una pausa del recolector de basura o del sistema pesa más sobre el promedio. De hecho, en casi todos los métodos $n=10$ sale más lento que $n=100$ con el mismo número de repeticiones, la firma típica de una JVM sin calentar y no de que el método sea realmente más costoso ahí.

4. Complejidad Teórica

4.1 List: Complejidad Teórica

método	SinglyNoTail	SinglyWithTail	DoublyNoTail	DoublyWithTail
pushFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
pushBack	$O(n)$	$O(1)$	$O(n)$	$O(1)$
popFront	$O(1)$	$O(1)$	$O(1)$	$O(1)$
popBack	$O(n)$	$O(n)$	$O(n)$	$O(1)$
find	$O(n)$	$O(n)$	$O(n)$	$O(n)$
erase	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addBefore	$O(n)$	$O(n)$	$O(1)$	$O(1)$
addAfter	$O(1)$	$O(1)$	$O(1)$	$O(1)$

4.2 MyStack y MyQueue: Complejidad Teórica

operación	Arreglo dinámico	Arreglo circular
push / enqueue	$O(1)$ amortizado	$O(1)$ amortizado
pop / dequeue (Stack)	$O(1)$	$O(1)$
dequeue (Queue)	$O(n)$	$O(1)$
peek / front	$O(1)$	$O(1)$
delete(valor)	$O(n)$	$O(n)$

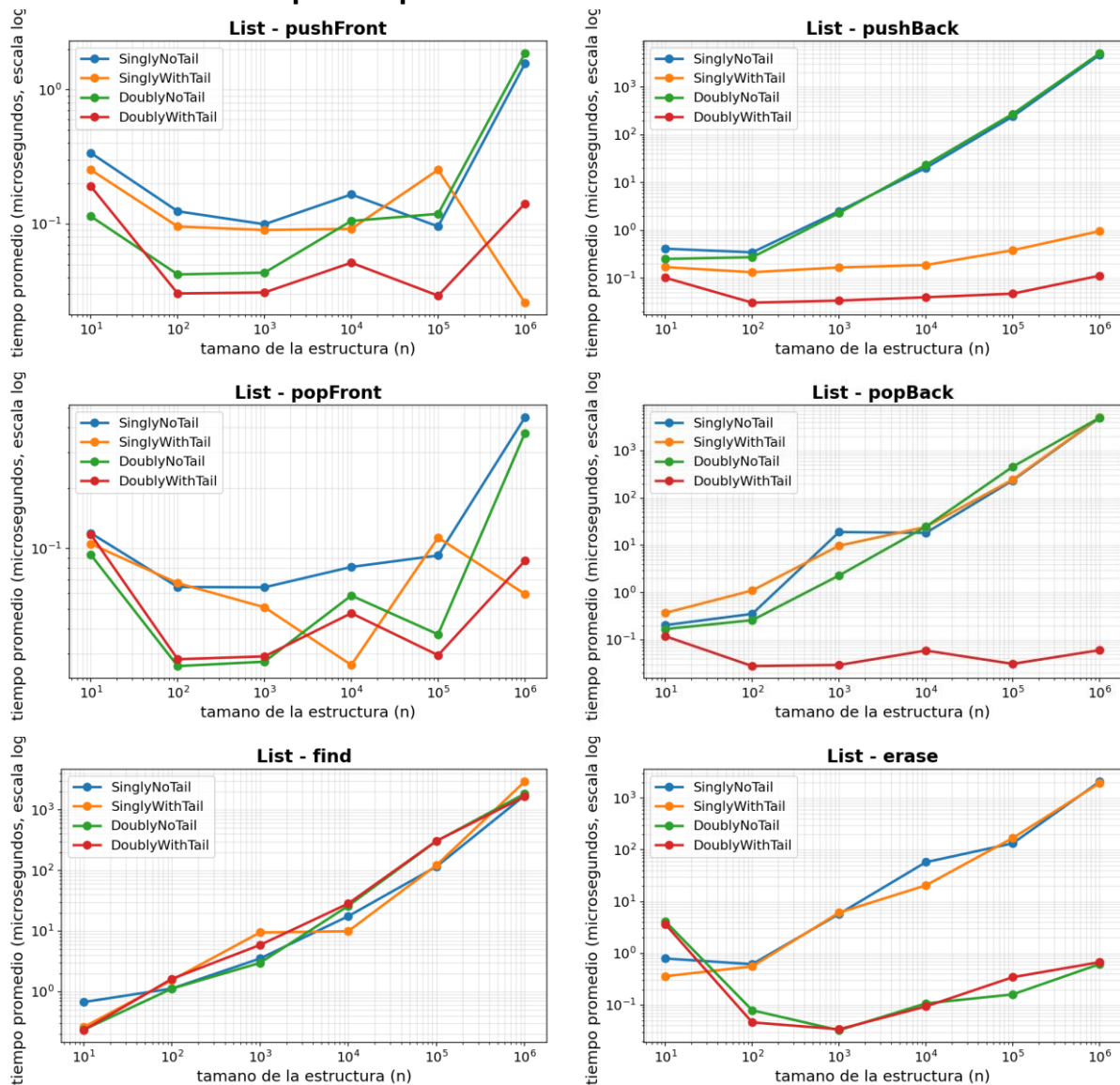
5. Hipótesis

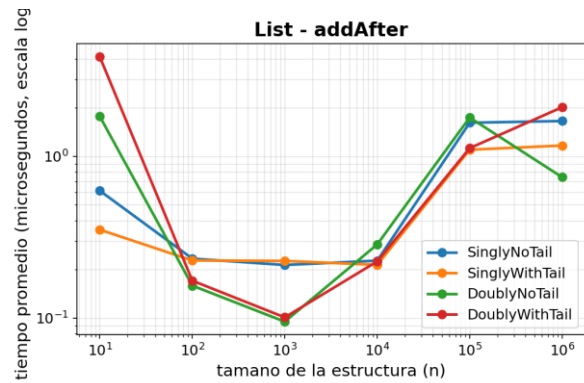
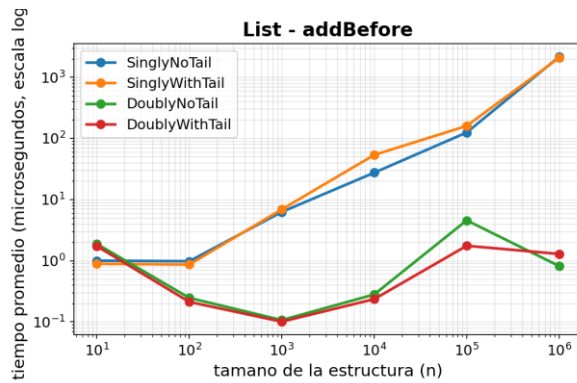
Hipótesis sobre List: antes de correr el benchmark, con solo mirar la tabla de arriba ya me imaginaba qué iba a pasar: pushFront, popFront y addAfter deberían mantenerse casi planos ($O(1)$) en las 4 variantes por igual; pushBack y popBack solo rápidos en las versiones con tail (o con prev, para popBack en DoublyWithTail), y creciendo en las demás; y find lento ($O(n)$) en las 4, porque ninguna variante indexa directo. Ahora toca ver qué tanto le atiné comparando esto con lo medido.

Hipótesis sobre MyStack y MyQueue: acá la hipótesis era más sencilla: esperaba que push, pop y peek dieran prácticamente igual entre el arreglo dinámico y el circular, porque los dos trabajan sobre el mismo extremo y son $O(1)$. Donde sí esperaba diferencia clara era en dequeue de Queue: $O(n)$ en el dinámico, porque hay que correr todo el arreglo al sacar por el frente, contra $O(1)$ en el circular, donde solo se mueve el índice.

6. Resultados

6.1 List: Resultados Empíricos por Método





SinglyLinkedListNoTail

n	10	100	1000	10000	100000	1000000
pushFront	0.34	0.12	0.10	0.17	0.10	1.56
pushBack	0.41	0.35	2.52	20.25	240.65	4664.91
popFront	0.12	0.06	0.06	0.08	0.09	0.45
popBack	0.21	0.36	19.08	18.34	235.37	4923.88
find	0.68	1.13	3.53	17.61	115.51	1712.92
erase	0.80	0.62	5.78	57.97	133.73	2086.53
addBefore	1.00	0.98	6.25	27.42	124.48	2146.79
addAfter	0.61	0.23	0.21	0.23	1.61	1.65

SinglyLinkedListWithTail

n	10	100	1000	10000	100000	1000000
pushFront	0.25	0.10	0.09	0.09	0.25	0.03
pushBack	0.17	0.13	0.17	0.19	0.38	0.96
popFront	0.11	0.07	0.05	0.03	0.11	0.06
popBack	0.37	1.11	9.80	24.48	245.54	5046.88
find	0.26	1.56	9.51	9.95	121.69	2907.83
erase	0.36	0.56	6.13	20.61	169.43	1948.91
addBefore	0.89	0.86	6.89	53.51	159.46	2068.60
addAfter	0.35	0.23	0.23	0.21	1.10	1.17

DoublyLinkedListNoTail

n	10	100	1000	10000	100000	1000000
pushFront	0.11	0.04	0.04	0.11	0.12	1.86
pushBack	0.25	0.27	2.30	23.47	273.03	5024.43
popFront	0.09	0.03	0.03	0.06	0.04	0.37
popBack	0.17	0.26	2.30	24.80	458.77	4992.75
find	0.24	1.13	2.99	26.05	303.68	1838.96
erase	4.13	0.08	0.03	0.11	0.16	0.62
addBefore	1.88	0.25	0.11	0.28	4.56	0.82
addAfter	1.77	0.16	0.09	0.29	1.74	0.74

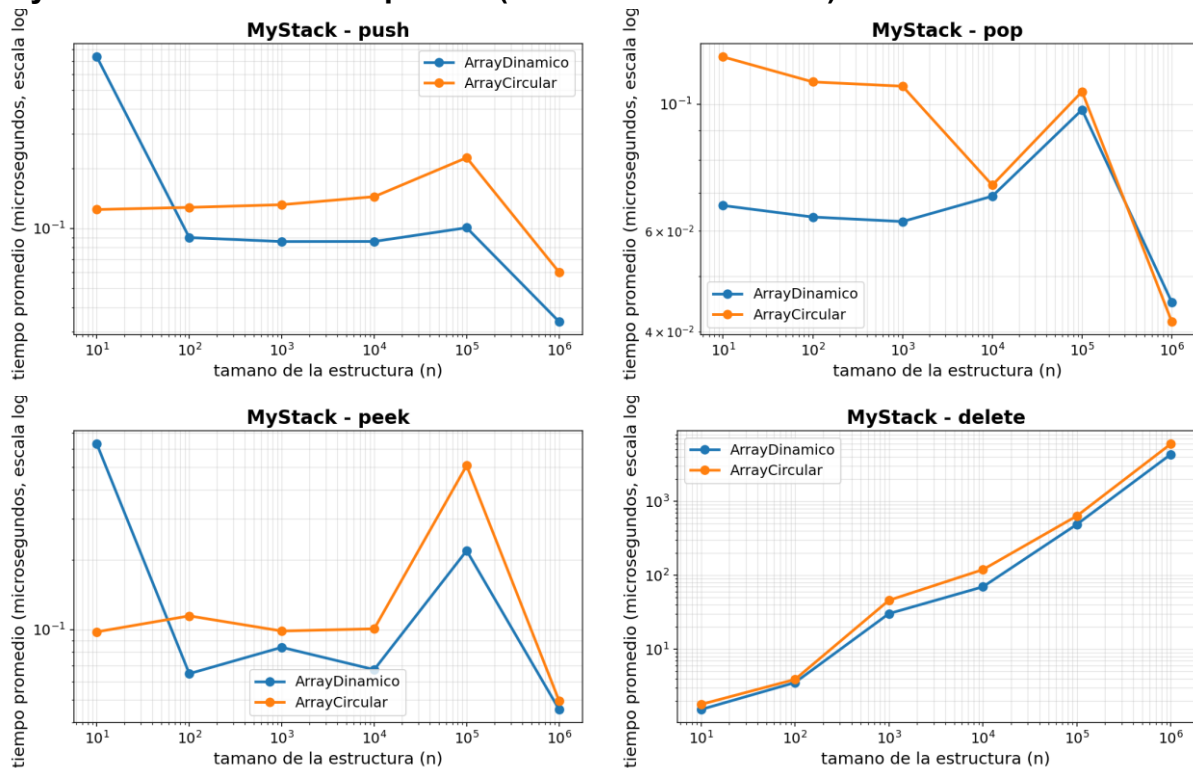
DoublyLinkedListWithTail

n	10	100	1000	10000	100000	1000000
---	----	-----	------	-------	--------	---------

pushFront	0.19	0.03	0.03	0.05	0.03	0.14
pushBack	0.10	0.03	0.03	0.04	0.05	0.11
popFront	0.12	0.03	0.03	0.05	0.03	0.09
popBack	0.12	0.03	0.03	0.06	0.03	0.06
find	0.23	1.63	5.91	28.45	306.48	1647.81
erase	3.62	0.05	0.03	0.09	0.35	0.67
addBefore	1.73	0.21	0.10	0.23	1.75	1.29
addAfter	4.13	0.17	0.10	0.22	1.12	2.01

Comparando esto con la hipótesis: en general sí se cumplió el patrón esperado, sobre todo en n grande (10^5 y 10^6), donde las curvas $O(n)$ (pushBack sin tail, popBack sin prev, find, erase/addBefore sin prev) se despegan clarísimo de las que quedan planas: pushBack en SinglyNoTail pasa de $0.41 \mu s$ en $n=10$ a $4664.91 \mu s$ en $n=1\ 000\ 000$, mientras que en DoublyWithTail se mantiene en $0.11 \mu s$. Donde no coincidió tan limpio fue en los tamaños chicos: varios métodos en teoría $O(1)$ -erase, addBefore y addAfter en las variantes Doubly- muestran picos en $n=10$ que no tienen que ver con la complejidad sino con lo ya explicado sobre repeticiones y calentamiento de JVM. También hay un salto raro en addBefore de DoublyLinkedListNoTail ($0.28 \mu s$ en $n=10\ 000$, $4.56 \mu s$ en $n=100\ 000$, $0.82 \mu s$ de vuelta en $n=1\ 000\ 000$) que no debería estar si la operación es $O(1)$; lo dejo anotado en la sección 8 (Análisis de Anomalías).

6.2 MyStack: Resultados Empíricos (Dinámico vs Circular)



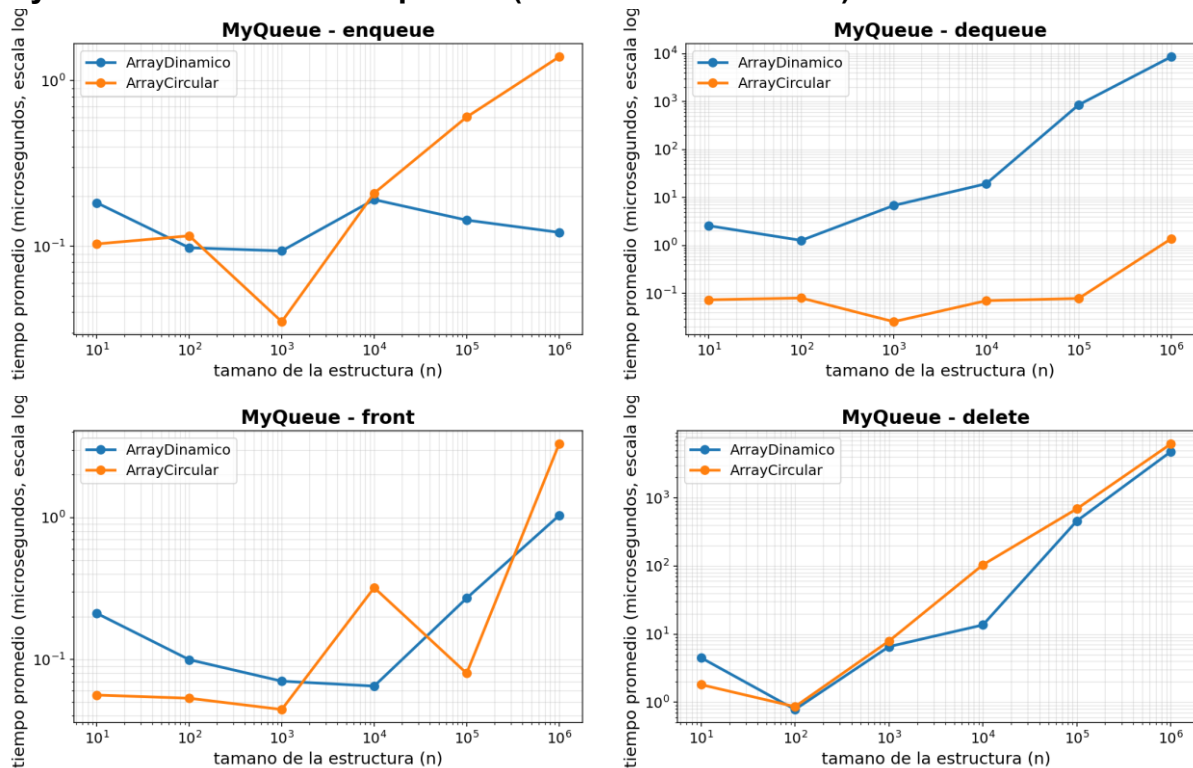
Arreglo Dinámico

n	10	100	1000	10000	100000	1000000
push	0.73	0.09	0.09	0.09	0.10	0.03
pop	0.07	0.06	0.06	0.07	0.10	0.05
peek	0.63	0.06	0.08	0.07	0.22	0.05
delete	1.53	3.54	30.27	69.68	486.63	4303.57

Arreglo Circular

n	10	100	1000	10000	100000	1000000
push	0.12	0.13	0.13	0.14	0.23	0.06
pop	0.12	0.11	0.11	0.07	0.11	0.04
peek	0.10	0.11	0.10	0.10	0.51	0.05
delete	1.80	3.90	45.82	119.03	632.77	5943.94

6.3 MyQueue: Resultados Empíricos (Dinámico vs Circular)



Arreglo Dinámico

n	10	100	1000	10000	100000	1000000
enqueue	0.18	0.10	0.09	0.19	0.14	0.12
dequeue	2.58	1.28	6.83	19.41	858.27	8594.46
front	0.21	0.10	0.07	0.06	0.27	1.03
delete	4.50	0.78	6.55	13.63	459.82	4778.73

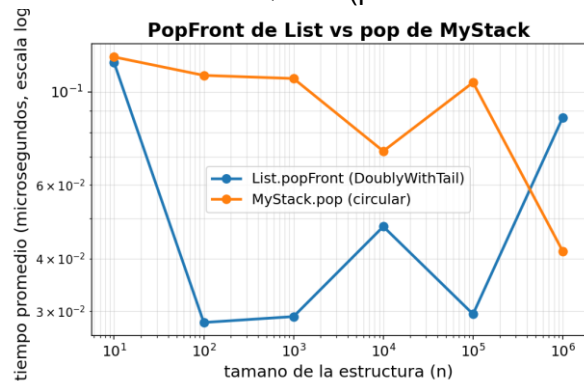
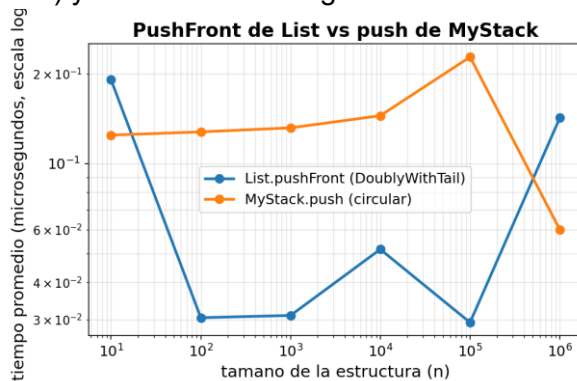
Arreglo Circular

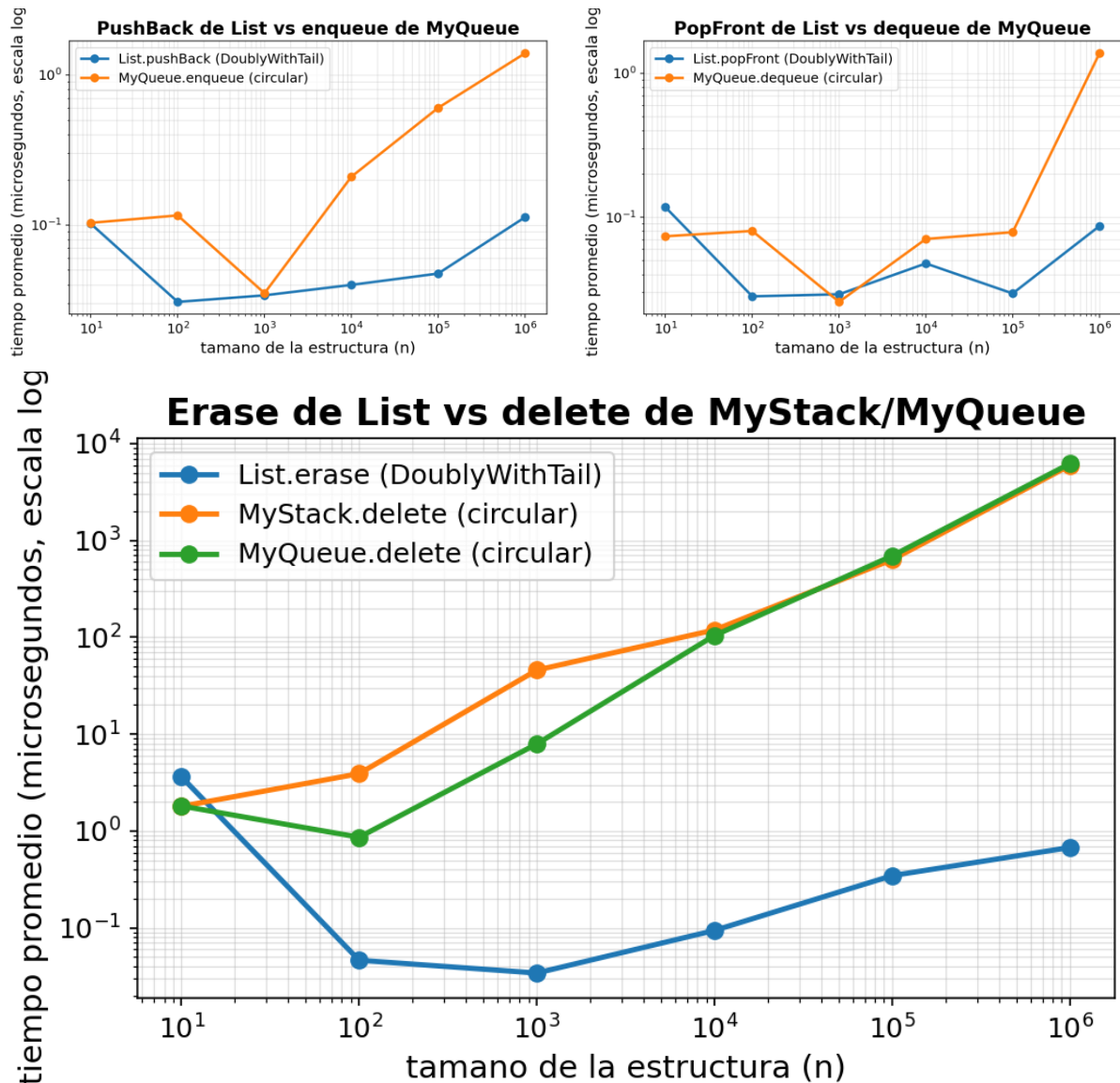
n	10	100	1000	10000	100000	1000000
enqueue	0.10	0.12	0.04	0.21	0.60	1.39
dequeue	0.07	0.08	0.03	0.07	0.08	1.38
front	0.06	0.05	0.04	0.32	0.08	3.30
delete	1.81	0.86	7.92	104.30	692.92	6209.17

Comparando con la hipótesis: en push/pop/peek del Stack, y en enqueue/front del Queue, los tiempos de las dos implementaciones se mantienen bajos y sin tendencia clara de crecimiento, como se esperaba de operaciones $O(1)$. La diferencia que predecía la teoría sí se ve en dequeue de Queue: ArrayDinamico pasa de 2.58 μs en $n=10$ a 8594.46 μs en $n=1\,000\,000$, mientras ArrayCircular se queda en 1.38 μs , justo la brecha entre $O(n)$ y $O(1)$ de la tabla teórica. En front de ArrayDinamico sí hay algo que no esperaba: sube a 1.03 μs en $n=1\,000\,000$ cuando en $n=10\,000$ estaba en 0.06 μs . Como esa medición se hizo con solo 15 repeticiones (la más baja de todas), sospecho que es más ruido que un problema real, pero no lo puedo descartar sin correrlo de nuevo con más repeticiones.

7. Comparación entre Implementaciones

Comparativa de Métodos Equivalentes (List vs MyStack/MyQueue) Para esta comparación tomé a DoublyLinkedListWithTail como representante de List (por ser la más completa de las 4) y la versión de arreglo circular como representante de Stack/Queue (por la misma razón).





- **Push/Pop:** En operaciones puras de apilar y desapilar las estructuras quedan prácticamente empatadas, al fin y al cabo todas son $O(1)$ para esto.
- **Delete / Erase:** Acá sí hay diferencia real: en eliminación por valor, List le gana a Stack/Queue basados en arreglos. Las dos son $O(n)$ para encontrar el valor, eso no cambia, pero en la lista enlazada, una vez encontrado el nodo, eliminarlo es $O(1)$; en un arreglo toca correr los elementos que quedan para cerrar el espacio.

8. Conclusiones

- **Listas Enlazadas:** Con todo esto medido, queda claro que ninguna implementación de List gana en todo. SinglyLinkedListNoTail es la más fácil de programar, pero no escala en las operaciones que tocan el final: pushBack pasa de 0.41 μs en $n=10$ a 4664.91 μs en $n=1\ 000\ 000$. Si lo que se busca es eficiencia $O(1)$ en ambos extremos, no tiene vuelta de hoja: toca

tener doble enlace y puntero a tail al mismo tiempo, o sea DoublyLinkedListWithTail, que en ese mismo pushBack se queda en apenas 0.11 μ s incluso con un millón de elementos. Tener solo el puntero tail (SinglyLinkedListWithTail) no alcanza para resolver popBack sin enlaces hacia atrás: ahí sigue en 5046.88 μ s en $n=1\ 000\ 000$, prácticamente igual de lento que la versión sin tail.

- **Arreglo Circular vs Dinámico:** El arreglo circular deja mucho mejor parada a la Cola: dequeue en ArrayDinamico llega a 8594.46 μ s en $n=1\ 000\ 000$, contra 1.38 μ s en ArrayCircular. En el Stack la diferencia es mínima (push y pop rondan valores parecidos en las dos versiones), porque ahí las dos ya operaban en $O(1)$ desde el arreglo plano y el circular no tenía mucho que mejorar.
- **Arreglos vs Listas:** Los arreglos siguen ganando en localidad de memoria y costo por elemento sobre extremos fijos. Comparando erase de List (DoublyWithTail) contra delete de MyStack y MyQueue en la sección 7, la lista se mantiene por debajo de 1 μ s incluso en $n=1\ 000\ 000$, mientras que delete en las estructuras de arreglo sube a varios miles de microsegundos, porque además de encontrar el valor ($O(n)$ en las tres) el arreglo tiene que correr los elementos para cerrar el espacio. Si lo que se necesita es insertar o eliminar en medio de la estructura sin desplazar memoria, ahí las listas enlazadas son la mejor opción.

9. Uso de Inteligencia Artificial

En este trabajo usé inteligencia artificial, específicamente Claude (Anthropic), como apoyo para organizar mejor el documento final y la estructura del repositorio en GitHub. El código de las cuatro implementaciones de List, de MyStack y MyQueue, los benchmarks, las pruebas y el análisis/conclusiones los hice yo a mano. Las gráficas también las generé yo con Python (matplotlib), a partir de los datos de los benchmarks; lo único que hizo la IA con las imágenes fue acomodarlas dentro del documento de Word. En general calculo que el apoyo de la IA estuvo alrededor de un 35%, porque el análisis y los procedimientos/requerimientos que pedía el taller los resolví yo. Todo el contenido generado con ayuda de IA fue revisado y editado por mí para que la información fuera precisa.